

Concept 6: Event-Driven vs Batch vs Streaming

Why This Matters

When you build a system that needs to **react to changes** (like checking deadlines, sending notifications, or processing data), you must choose HOW the system detects and responds to those changes. There are three fundamental paradigms, and choosing the wrong one adds unnecessary complexity.

The Three Paradigms

1. Event-Driven

Principle: Something happens → an immediate reaction is triggered.

How it works:

- A **producer** emits an event (e.g., "RFQ moved to Technical Review stage")
- An **event bus/broker** receives and distributes the event (e.g., RabbitMQ, Kafka, Redis Pub/Sub)
- A **consumer/listener** is subscribed and reacts in real-time

Mental model: A doorbell. Someone presses it → you hear it instantly → you react.

Example in our context:

RFQ stage changes → event fires → listener checks "should I send a reminder?" → sends notification

Strengths:

- Real-time reactivity (near-instant response)
- Natural fit when the trigger IS an action/event
- Each service only reacts to events it cares about

Weaknesses:

- **Coupling:** The reminder service must be connected to the event bus. If the bus goes down, events are lost (unless using persistent queues)
- **Infrastructure complexity:** You need a message broker (RabbitMQ, Kafka), event schemas, dead letter queues, retry logic
- **Debugging difficulty:** Following the flow of "what triggered what" across distributed events is harder than reading a sequential script
- **Ordering guarantees:** Events may arrive out of order without careful design

Best for: User actions that need immediate response, real-time notifications, service-to-service communication where latency matters.

2. Batch Processing

Principle: A process runs at regular intervals, examines the current state, and acts.

How it works:

- A **scheduler** triggers a script/process every N minutes (cron job, in-app timer)
- The process **reads the current state** from the database
- It compares state against rules and **takes action** on matching records
- It doesn't care what happened between runs — only what the data looks like NOW

Mental model: A security guard doing rounds every 30 minutes. They don't react to every noise — they check every door on schedule.

Example in our context:

Every 5 minutes → batch reads all active reminders → compares trigger_date vs NOW → fires due reminders

Strengths:

- **Loose coupling:** Only needs database access, not a live connection to other services
- **Simple to implement:** A script with a loop or a cron job

- **Easy to debug:** You can re-run the batch manually, inspect the database, trace every decision
- **Resilient:** If the batch fails, it simply catches up on the next run (no lost events)
- **Idempotent by nature:** Checking "is this reminder due?" produces the same answer regardless of how many times you check

Weaknesses:

- **Latency:** There's a delay between "something changes" and "we detect it" (up to the interval duration)
- **Resource waste:** The batch runs even when there's nothing to process
- **Scaling concern:** If the dataset grows very large, each batch run takes longer

Best for: Deadline checking, periodic reports, data cleanup, any process where a few minutes of delay is acceptable.

3. Streaming

Principle: A continuous flow of data is processed in near-real-time, often with state management.

How it works:

- Data flows continuously through a **stream** (e.g., Kafka Streams, Apache Flink)
- Processing happens on the fly: filtering, aggregating, windowing
- The system maintains **state** across the stream (counters, windows, running totals)

Mental model: A conveyor belt in a factory. Items flow continuously, and workers process them as they pass by, keeping track of counts and quality metrics.

Example:

All RFQ events flow into a stream → the stream processor aggregates "number of RFQs per client this month" → updates a dashboard in real-time

Strengths:

- Handles high-volume, continuous data efficiently

- Can do windowed aggregations (e.g., "average response time in the last hour")
- Near-real-time with state management

Weaknesses:

- **Most complex** to implement and operate
- Requires specialized infrastructure (Kafka + stream processor)
- Overkill for simple use cases

Best for: High-volume data pipelines, real-time analytics, IoT sensor data, log processing.

Comparison Table

Criteria	Event-Driven	Batch	Streaming
Latency	Real-time (ms)	Periodic (minutes)	Near-real-time (seconds)
Coupling	High (needs event bus)	Low (only needs DB)	Medium (needs stream infra)
Complexity	Medium	Low	High
Infrastructure	Message broker	Cron/scheduler	Stream platform
Debugging	Hard (distributed)	Easy (sequential)	Medium
Failure recovery	Need dead letter queues	Auto-catches up next run	Checkpoints/offsets
Best trigger	"Something just happened"	"Is anything due?"	"What's happening overall?"

Why Batch Wins for Our Reminder Use Case

The reminder system asks: **"Are there any reminders whose deadline has passed?"**

This is fundamentally a **state comparison** (trigger_date vs. current_time), not a reaction to an event. The answer to this question is the same whether you check it

via event, batch, or stream. But:

1. **No real-time need:** A 5-minute delay in sending a reminder is perfectly acceptable
2. **Minimal coupling:** The batch just reads from the reminder table — no event bus dependency
3. **Simple implementation:** A cron job or in-app loop is all we need
4. **Easy testing:** Run the batch manually, inspect results, verify logic

Event-driven would work but adds infrastructure (message broker) and complexity (event schemas, consumer management) that provide zero benefit for checking deadlines.

Key Vocabulary

Term	Definition
Producer	The component that creates/emits an event
Consumer/Listener	The component that receives and reacts to an event
Message Broker	Infrastructure that routes events between producers and consumers (RabbitMQ, Kafka)
Cron Job	An OS-level scheduler that runs scripts at defined intervals
Idempotent	An operation that produces the same result no matter how many times you run it
Dead Letter Queue	Where failed/unprocessable events go for later investigation
Windowing	In streaming, grouping events by time windows for aggregation